أكاديمية طويق
Tuwaiq Academy

وجهتنا

Wejhatna

AI-Powered Smart & Accessible Riyadh Trip Planner

AI Champions Challenge 2026

Project Documentation

Team Members

Nora Alkhudair

Aryam Almutairi

Shahad Alotaibi

Jowaher Almutairi

August 2026

Table of Contents

1. Executive Summary

Wejhatna (وجهتنا) is a smart trip-planning platform focused on Riyadh. Instead of producing a generic list of attractions, the system builds a personalized day-by-day itinerary around the traveler's real constraints: trip date and duration, group size, children or elderly travelers, accessibility requirements, budget, interests, preferred daily start/end times, prayer times, weather conditions, route distance, travel time, and traffic context.

The solution combines an AI planning agent powered by Gemini with structured local data, spatial database capabilities, web search, and live/structured tools. The agent uses these sources to select suitable places, organize them into a realistic schedule, and explain why each activity was chosen. The final web interface presents the plan on a single page and supports light/dark themes. A separate replan flow allows the itinerary to be regenerated when conditions or user needs change.

> **Core idea:** Move from "recommend places" to "build an executable trip plan" by combining AI reasoning with real constraints and external data.

2. Problem and Motivation

Planning a city trip usually requires the traveler to switch between maps, weather services, prayer-time sources, attraction pages, transport information, and accessibility details. Generic recommendation systems may suggest good places but often ignore whether the plan is practical at a specific date and time.

- Recommendations may be geographically scattered, increasing travel time.
- Future traffic is uncertain and must not be presented as guaranteed live traffic.
- Outdoor activities can be unsuitable in extreme heat or poor weather.
- Prayer times affect the realistic timing of activities in Riyadh.
- Families with children, elderly travelers, or mobility needs require different pacing and venue choices.
- Opening hours, visit duration, route distance, and transport access must be considered together.

3. Project Objectives

- Generate personalized multi-day itineraries for Riyadh.
- Use Gemini as an agent that can call tools rather than relying only on free-form chat.
- Combine local database records with web/live sources when needed.
- Calculate route distance and travel time between consecutive activities; the first activity is calculated from Riyadh City Center.
- Represent traffic carefully: use live information when available, otherwise provide an explicit probability/estimate based on place, date, day, and time.
- Respect all five daily prayer times in schedule construction.
- Adapt the itinerary to children, elderly travelers, and stated accessibility requirements.
- Provide transparent reasons for selected activities and contextual planning notes.
- Allow replanning when the user reports a changed condition such as heavy traffic or excessive walking.

4. Target Users and User Inputs

The primary user is a visitor or resident planning leisure activities in Riyadh. The form intentionally collects a small set of high-impact constraints so the AI agent can produce a useful plan without a long setup process.

Input	Purpose
Trip date	Anchors prayer times, date-specific planning and future/live data selection.
Number of days (1-30)	Controls itinerary duration.
Number of people	Supports group-aware planning.
Children	Encourages family-suitable pacing and places.
Elderly	Encourages lower-strain scheduling and accessibility awareness.
Accessibility requirements	Free-text needs such as wheelchair-friendly or low walking.
Budget	Low, medium, or high planning preference.
Interests	Museums, parks, heritage, shopping, events, dining, etc.
Preferred start/end time	Defines the daily planning window.

5. Solution Overview and User Journey

The user completes one planner form. The frontend sends the request to the Flask backend. The backend passes the request to the AI agent. The agent interprets the constraints, retrieves structured information through tools, constructs the itinerary, and returns structured JSON. The frontend then renders weather, prayer, traffic, route and activity information as day cards.

- 1) User enters trip preferences.
- 2) Frontend sends a POST request to /api/plan-trip.
- 3) Flask calls create_itinerary(payload).
- 4) Gemini agent evaluates constraints and invokes available tools.
- 5) Tools query the PostgreSQL/PostGIS database and external services as required.
- 6) Agent returns a structured itinerary.
- 7) JavaScript renders the result in the same page.
- 8) If circumstances change, the replan flow can send the current plan plus a reason to /api/replan.

6. System Architecture

Layer	Role
Frontend	Single-page HTML/CSS/JavaScript interface; captures user inputs and renders the itinerary.
Web/API Layer	Flask application exposing planning and replanning endpoints.
AI Agent	Gemini-based orchestration and reasoning layer; selects and calls tools and produces structured output.
Tool Layer	Python functions for database, web search, routing/traffic, weather, prayer-time and related lookups.
Data Layer	PostgreSQL 17 with PostGIS for structured and spatial Riyadh data.
External Services	Tavily web search and location/live-data services used when local data is insufficient or freshness is required.

Design principle: The language model does not act as the database. Structured facts are retrieved through tools; Gemini coordinates, reasons over them, and produces the final plan.

7. AI Agent Design and Tool Calling

The agent is implemented in Python and is separated from the web application. This keeps the planning logic independent from the interface. The project structure includes agent.py for itinerary generation/replanning,

tools.py for callable capabilities, prompts.py for the system instructions and output rules, and config.py for environment/configuration values.

The system prompt constrains the model to return a predictable structured result. This is important because the frontend does not parse conversational prose; it expects fields such as destination, number_of_days, days, prayer_times, weather, traffic_summary, itinerary, route_to_activity and traffic_to_activity.

- Gemini performs orchestration and planning.
- Tool calls ground decisions in structured or externally retrieved information.
- The agent distinguishes estimated future conditions from known/live conditions.
- Structured JSON makes the AI output directly consumable by the frontend.

8. Data Sources and External Services

Source / Service	Use in Wejhatna
PostgreSQL + PostGIS	Local structured and geospatial data for points of interest, transit and other Riyadh planning data.
Tavily Web Search	Web retrieval for information that is not available or sufficiently current in the local database.
Gemini API	AI reasoning, tool orchestration and structured itinerary generation.
TomTom Routing / Traffic	Routing and traffic capability investigated/integrated for route distance, travel time and traffic context.
Weather tool/service	Provides weather information used to decide indoor/outdoor suitability; future values are labeled as estimates when applicable.
Prayer-time tool/data	Provides the five daily prayer times used by the schedule.
Google AI Studio	Used for the early interactive prototype and interface exploration.

Important implementation note: configuration evidence also showed experimentation with Waze credentials. Because authorization/integration issues were encountered, the documentation does not claim Waze as a dependable production data source. The final planner is designed to degrade gracefully by labeling unavailable traffic as unknown and using explicit probability-based context rather than inventing live data.

9. Database Design

The project uses a PostgreSQL database named wejhatna with the PostGIS extension enabled. PostGIS adds spatial data types and functions, allowing locations to be stored as geometry and enabling distance/proximity queries that are more reliable than treating coordinates as ordinary text or numbers.

The implemented schema includes structured Riyadh data such as points of interest and transport information. Project screenshots verify tables including bus_stops, events, metro_lines, metro_stations, prayer_times, saudi_holidays, transit_routes and pois. A metro_station_lines relationship table was also used during data import/schema work.

Table / Data Group	Purpose
pois	Points of interest used as candidate attractions; accessibility fields include wheelchair_accessible.
metro_stations	Riyadh Metro station locations and metadata.
metro_lines / metro_station_lines	Metro line structure and station-line relationships.
bus_stops	Bus stop locations for nearby transit context.
transit_routes	Structured transit route information.
events	Event-related records for date-aware recommendations.
prayer_times	Prayer-time data available to the planning layer.
saudi_holidays	Holiday context that can affect planning.

Data preparation involved importing spreadsheet/CSV-derived records into PostgreSQL, validating table counts and column types, checking coordinates, and correcting import/schema mismatches. Spatial fields such as geom and flags such as spatial_review_required were used to support geospatial validation.

10. Routing, Travel Time and Traffic Logic

Travel time is not generated as an arbitrary number. Each activity includes route information describing the origin, distance in kilometers, travel time in minutes, and route source. For the first activity of each day, the route begins from Riyadh City Center. Subsequent activities are calculated from the previous activity, which makes the itinerary sequential and geographically coherent.

Traffic is handled conservatively. When real traffic is unavailable - especially for future dates - the output can set traffic_type and traffic_level to unknown while still providing a congestion_probability and explanation based on the destination area, date/day and time. This avoids presenting a prediction as live fact.

Field	Meaning
distance_km	Estimated/returned route distance between origin and activity.
travel_time_minutes	Route travel duration used to create realistic gaps between activities.
traffic_type	Indicates whether traffic information is live/known or unknown/estimated.
congestion_probability	Low, moderate or high probability when exact traffic is unavailable.
traffic_delay_minutes	Expected additional delay when supported by the planning logic.
traffic_reason	Human-readable explanation such as rush hour, weekend venue traffic or a short local trip.

11. Weather and Prayer-Time Planning

Weather and prayer times are planning constraints rather than decorative information. Each day contains a weather object with forecast type, summary, minimum/maximum temperature and a planning note. When the trip date is too far in the future for a reliable forecast, the system labels the result as estimated instead of implying real-time accuracy.

Prayer times include Fajr, Dhuhr, Asr, Maghrib and Isha. The agent uses these times to avoid unrealistic scheduling conflicts and to create natural breaks in the itinerary. This makes the plan better aligned with the local Riyadh travel context.

12. Accessibility-Aware Planning

Accessibility is a first-class input. The user can indicate elderly travelers, children and free-text accessibility requirements. The database schema also includes accessibility-related POI information such as wheelchair accessibility. The agent uses these constraints to favor suitable places, reasonable visit durations, lower walking burden where requested, and practical transport choices.

- Wheelchair/accessibility information can be used when available in structured data.

- Elderly and children inputs influence pacing and venue suitability.
- Accessibility requirements remain visible to the agent throughout planning.
- The agent provides a reason for each selected activity, improving transparency.

13. Frontend and User Experience

The final web UI is a custom single-page interface built specifically for Wejhatna. The user enters all trip constraints on the left-side planner and receives the generated itinerary on the same page. The interface uses the Wejhatna teal identity, responsive cards, day-by-day timelines, status/insight sections and a dark-mode toggle.

- HTML template rendered by Flask.
- Custom CSS for the Wejhatna visual identity and responsive layout.
- JavaScript for form state, API calls, loading/error states and itinerary rendering.
- Light/dark mode; selected theme is stored in localStorage.
- Result cards organize weather, prayer times, traffic and activities without exposing raw JSON to the user.

14. Backend and API Contract

Flask acts as the bridge between the browser and the AI agent. The root route renders the interface. The planning endpoint accepts JSON and passes it to create_itinerary. The replanning endpoint passes the request to replan_itinerary. Exceptions are returned as JSON errors with HTTP status 500 so the frontend can show an error state.

Endpoint	Method	Purpose
/	GET	Render the single-page Wejhatna interface.
/api/plan-trip	POST	Generate a new itinerary from user constraints.
/api/replan	POST	Regenerate/adjust an existing plan based on a changed condition or user reason.

The planning request includes destination, date, number_of_days, number_of_people, children, elderly, accessibility_requirements, budget, interests, preferred_start_time and preferred_end_time. The UI was extended to accept trips from 1 to 30 days.

15. Replanning Capability

Replanning is designed as a separate capability so it does not complicate the initial planner. After a plan exists, the user can provide a reason such as “heavy traffic,” “too much walking,” or “change the next activity.” The frontend can send that context to /api/replan, where replan_itinerary can ask the agent to reconsider the plan while preserving relevant trip constraints.

16. Prototype

Before final integration, an interactive prototype was created in Google AI Studio to validate the visual direction and user journey. The prototype explores destination/area selection, preferred attractions, accessibility choices and a branded Riyadh travel experience. It is intentionally treated as an early UX concept rather than the final backend-connected implementation.

Prototype link: https://aistudio.google.com/apps/b39b4bba-7f2f-4c0c-8b92-b80fea332ad6?project=gen-lang-client-0530250805&showPreview=true&pli=1&showAssistant=true

The prototype configuration uses React + Vite with TypeScript and Tailwind CSS and includes Google Maps-related dependencies and Google GenAI support. The final integrated UI uses Flask templates with HTML/CSS/JavaScript and connects directly to the Python agent backend.

17. Technology Stack

Category	Technologies
AI	Gemini / google-genai; prompt-guided structured output; tool calling.
Backend	Python, Flask, requests, python-dotenv.
Agent/Search	Custom Python agent/tools; Tavily Python client.
Database	PostgreSQL 17, PostGIS, psycopg.
Frontend	HTML, CSS, JavaScript, Jinja/Flask templates; dark mode/localStorage.
Prototype	Google AI Studio, React, Vite, TypeScript, Tailwind CSS.
Location/Traffic	TomTom Routing and Traffic capability; spatial database queries.
Development	VS Code, pgAdmin, Git/GitHub.

18. Data Processing and Validation

The data workflow was not limited to storing a spreadsheet. Structured source data was prepared for relational/spatial use, imported into PostgreSQL, and checked after import. During development, failed imports and schema mismatches were debugged rather than ignored.

- Enabled PostGIS in the wejhatna database.
- Created the project schema and verified the expected tables.
- Imported data from prepared spreadsheet/CSV sources.
- Checked row counts across core tables.

- Inspected column names and PostgreSQL data types.
- Validated coordinate/spatial fields and used review flags where needed.
- Added/used accessibility-related POI fields such as wheelchair_accessible.

19. Output Structure

The agent response is structured around a destination, number of days and a days array. Each day contains date-specific context and an itinerary. This contract lets the frontend render deterministic components even though the content itself is generated dynamically.

Object	Key contents
day	date, prayer_times, weather, traffic_summary, itinerary[]
prayer_times	fajr, dhuhr, asr, maghrib, isha
weather	forecast_type, summary, temperature_min_c, temperature_max_c, planning_note
traffic_summary	traffic_type, overall_level, congestion_probability, summary
activity	activity_id, place_id, name/type, coordinates, start/end, duration, route, traffic, reason
route_to_activity	from, distance_km, travel_time_minutes, route_source
traffic_to_activity	traffic_type, traffic_level, congestion_probability, delay, reason

20. Testing and Current Limitations

Development testing included local Flask execution, API-to-UI integration, database import/schema verification, tool output inspection, and external API debugging. The project also exposed practical limitations that are explicitly handled or documented rather than hidden.

- Future weather cannot be treated as a precise live forecast far in advance; estimated values must be labeled.
- Future traffic cannot be known exactly; probability-based congestion context is used when live data is unavailable.
- External APIs can fail because of credentials, authorization, quota or service availability.
- Some accessibility and opening-hour information may be incomplete and can require web verification.
- The current development server at 127.0.0.1:5000 is for local testing; public deployment requires a production host and environment variables.
- AI output remains dependent on source quality and tool availability; the system therefore uses structured fields and explicit unknown/estimated states.

21. Repository Organization

The GitHub repository is organized by responsibility so the final project is easy to review and maintain. The top level contains the main README and project folders such as Backend and AI Agent, Database, Fontend, docs and prototype.

Folder	Contents
Backend and AI Agent	Python backend/agent logic, tool functions, prompts, configuration and runtime requirements.
Database	PostgreSQL/PostGIS schema, prepared import data and database documentation.
Fontend	Final integrated web interface assets/templates.
docs	Project documentation and supporting material.
prototype	Early Google AI Studio prototype, README and screenshots.
README.md	Repository-level project overview and navigation.

22. Privacy, Security and Deployment Notes

- API keys and database credentials belong in .env/environment variables and must not be committed to GitHub.
- The repository uses an .env.example pattern to document required variables without exposing secrets.
- Production deployment should disable Flask debug mode and use an appropriate production WSGI/server environment.
- External service failures should be handled with clear error/unknown states rather than fabricated values.
- Database access should use least-privilege credentials in a deployed environment.

23. Future Enhancements

- Deploy the Flask application to a public production URL for judging and external use.
- Expand verified live routing/traffic integration and add graceful provider fallback.
- Add interactive map visualization for each day and route.
- Improve opening-hours verification and event freshness.
- Expand accessibility metadata and venue-level facilities.
- Add saved trips/user accounts if required beyond the challenge scope.
- Support Arabic in the final integrated interface in addition to English.
- Add automated tests for API contracts, tool functions and itinerary validation.

24. Conclusion

Wejhatna demonstrates an AI-agent approach to tourism planning in which the model is not used as a simple recommendation chatbot. The system combines Gemini reasoning, callable tools, a PostgreSQL/PostGIS data layer, web/live information and a dedicated web interface to create a plan that is personalized, geographically aware, prayer-aware, weather-aware, accessibility-aware and transparent about uncertainty. The project's strongest technical characteristic is the integration of multiple capabilities into one structured planning flow: the user provides one request, and the system coordinates the data and tools needed to return an actionable itinerary.

Project outcome: A working end-to-end foundation for a smart Riyadh itinerary planner, with a validated prototype, structured database, AI agent, backend API, integrated frontend and replanning path.